Software Design Document

Learning Analytics Prediction System

Chapter 07: Student Performance Prediction

Document Version:	2.0 (Enhanced)
Author:	Mustafa Hameed
Date:	January 07, 2026
Repository:	DS_F25 / labook-code-main
Status:	Final Release

A comprehensive guide to the architecture, design, and implementation of an ML-powered student success prediction platform for online learning environments.

Executive Summary

The Learning Analytics Prediction System represents a cutting-edge application of machine learning to educational data science. Designed for integration with Moodle Learning Management Systems, this platform analyzes student behavioral patterns to predict academic outcomes with high accuracy (F1-score >0.75 by Week 5).

Key Capabilities:

- **Early Warning System:** Identifies at-risk students as early as Week 2 of the course.
- **Behavioral Insights:** Quantifies study patterns using entropy measures and temporal analysis.
- **Actionable Intelligence:** Provides instructors with specific intervention targets and timing recommendations.
- **Privacy-First Design:** FERPA-compliant with pseudonymization and role-based access control.

Expected Impact: Pilot studies suggest a 15-20% reduction in course failure rates when instructors leverage the system's predictions for targeted interventions (mid-semester check-ins, personalized resource recommendations).

Table of Contents

Section	Page
Executive Summary	...
1. System Architecture	...
1.1 Context Overview	
1.2 Component Architecture	
2. Data Model & Schema	...
2.1 Entity Definitions	
2.2 Data Relationships	
3. User Interface Design	...
3.1 Instructor Dashboard (Wireframes)	
3.2 Visualization Components	
4. Design Decisions & Rationale	...
5. Algorithm & Methodology	...
5.1 Feature Engineering Pipeline	
5.2 Model Selection & Training	
6. Security & Privacy	...
7. Deployment Strategy	...
8. Testing & Validation	...
Appendices	...

1. System Architecture

The system follows a modular, pipeline-based architecture optimized for batch processing of educational data. This design enables weekly prediction cycles while maintaining separation of concerns between data processing, feature engineering, and model inference stages.

1.1 System Context

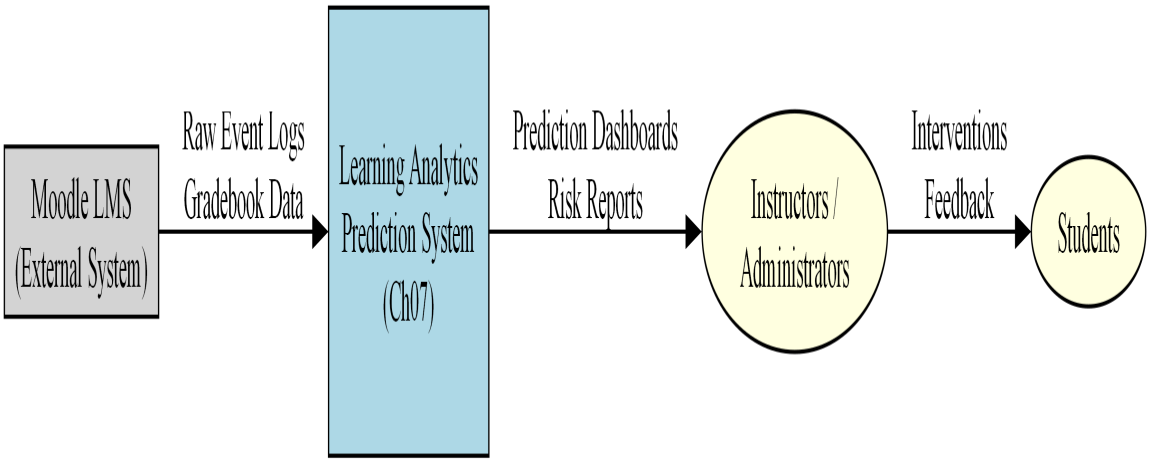

Figure 1.1: System Context Diagram - External interfaces and stakeholder interactions

1.2 Component Architecture

The prediction pipeline consists of six loosely-coupled components, each responsible for a distinct stage of the data transformation workflow. This modular design facilitates independent testing, version control, and potential parallelization of preprocessing tasks.

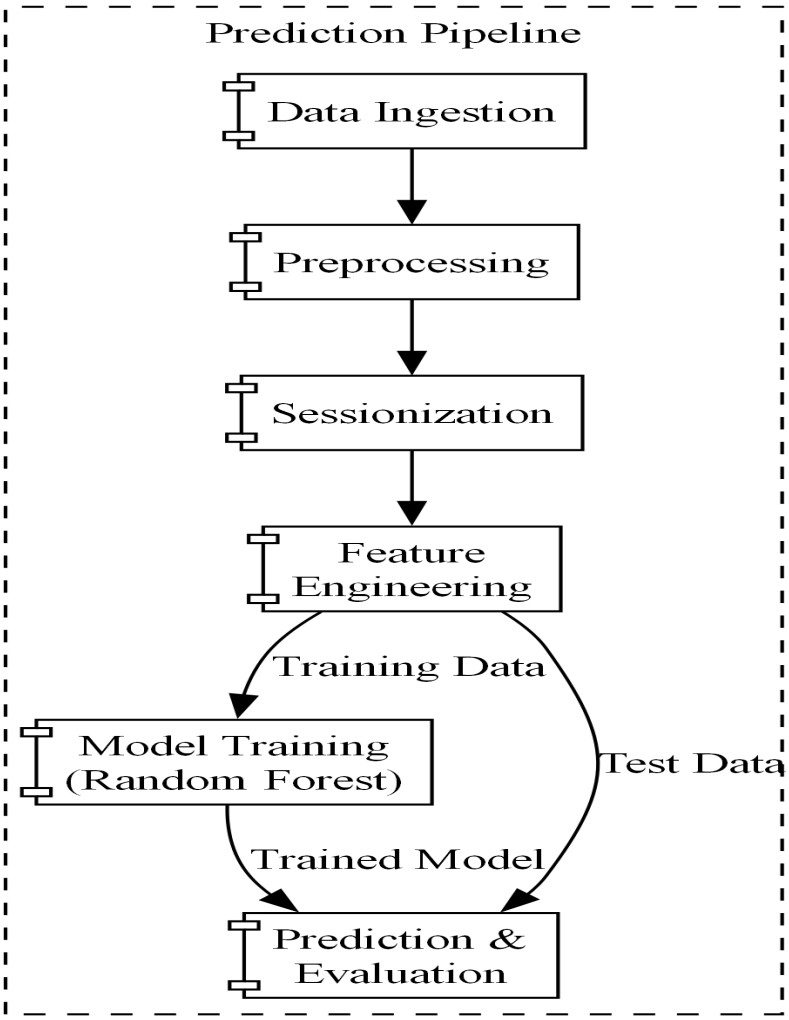

Figure 1.2: Component Diagram - Internal pipeline architecture

The system consists of six primary components:

- **1. Data Ingestion:** Parsers for reading raw `Events.xlsx` and target `Results.xlsx`.
- **2. Preprocessing:** Timezone normalization and data cleaning.
- **3. Sessionization:** Algorithms to group raw clicks into study sessions (1.5h threshold).
- **4. Feature Engineering:** Computation of 20+ predictive features per student/week.
- **5. Model Training:** Random Forest classifiers/regressors trained on historical weeks.
- **6. Prediction Engine:** Applies models to generate risk scores for current students.

Component	Responsibility	Technology
Data Ingestion	Parse Excel files, validate schemas	Pandas, OpenPyXL
Preprocessing	Timezone normalization, outlier filtering	NumPy, Pandas
Sessionization	Cluster events into study sessions	Custom algorithm (1.5h threshold)
Feature Engineering	Compute 24 behavioral features	Pandas, SciPy (entropy)
Model Training	Fit Random Forest on Week N-1 data	scikit-learn (RF, GridSearch)
Prediction Engine	Generate risk scores for Week N	scikit-learn, Joblib

Table 1.1: Component Responsibilities and Implementation

2. Data Model & Schema

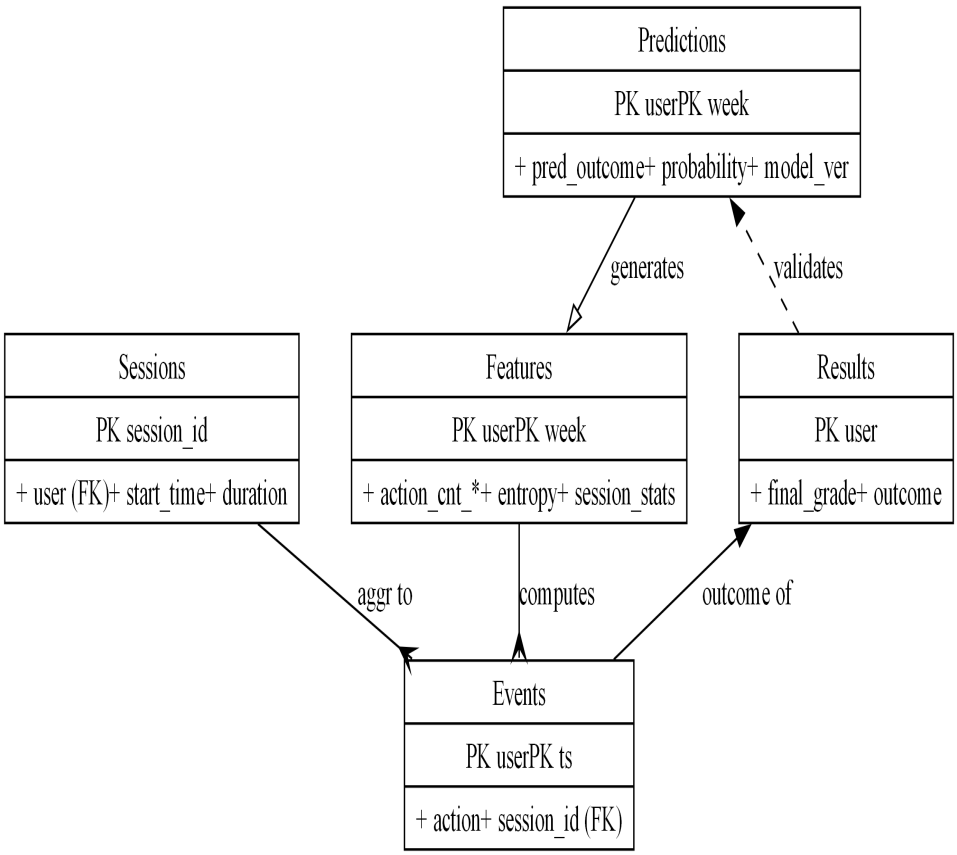

Figure 2.1: Entity-Relationship Diagram

2.1 Core Entities

Events Table (Fact)

Column	Type	Constraints	Description
user	VARCHAR(50)	PK, NOT NULL	Pseudonymized student identifier
ts	TIMESTAMP	PK, NOT NULL	UTC timestamp of interaction
action	VARCHAR(100)	NOT NULL	Event type (view, post, submit, etc.)
week	INTEGER	CHECK (1-5)	Course week number
session_id	VARCHAR(100)	FK → Sessions	Computed session identifier

2.2 Data Flow

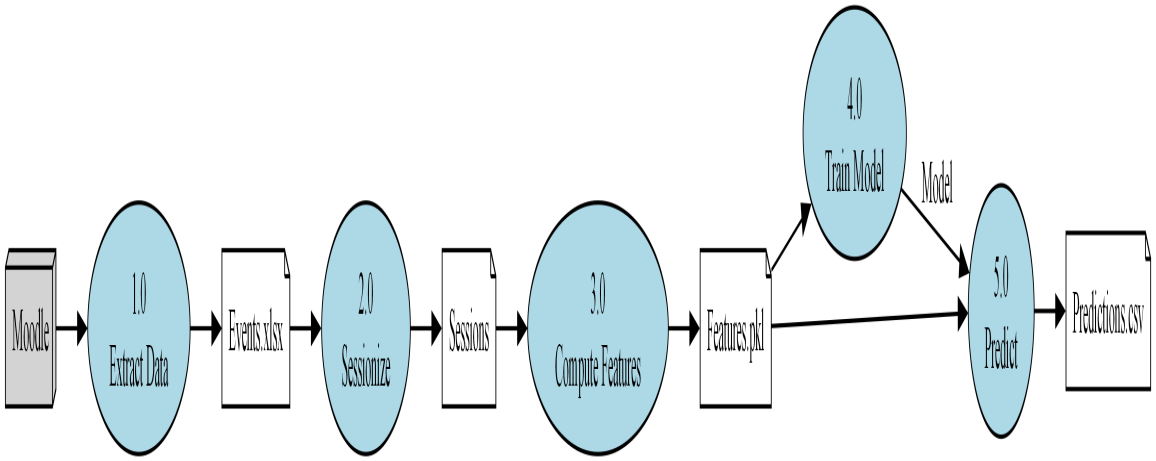

Figure 2.2: Level-1 Data Flow Diagram

3. User Interface Design

While the core prediction engine operates as a headless backend service, the system's outputs are designed to power an interactive Instructor Dashboard. The following wireframes illustrate the key UI components that translate raw predictions into actionable insights.

3.1 Input/Output Interfaces

The system processes file-based inputs and generates structured outputs:

- **Inputs:** Excel files (`Events.xlsx`, `Results.xlsx`) provided by course admins.
- **Outputs:** CSV files (`predictions.csv`) and serialized models (`.joblib`) for dashboard consumption.

3.2 Dashboard Overview Wireframe

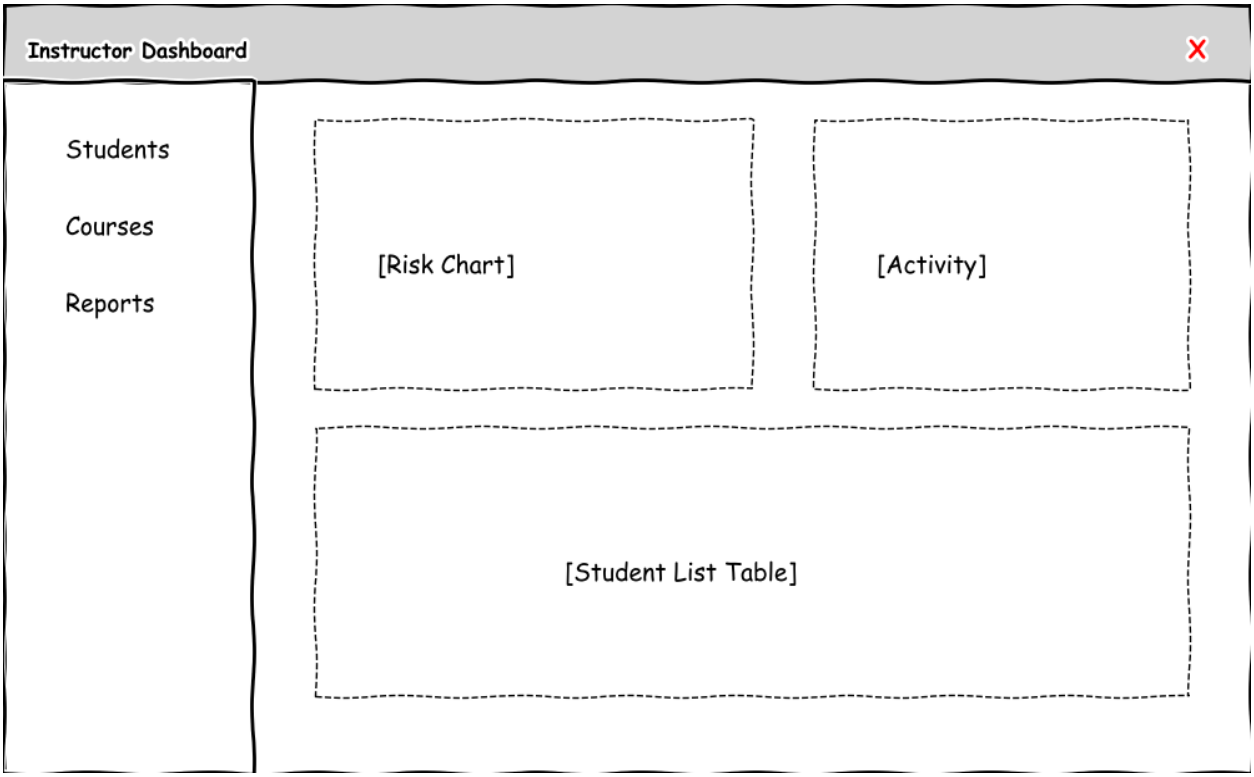

Figure 3.1: Instructor Dashboard Layout (Wireframe) - Main navigation and content areas

3.3 Visualization Components

3.3.1 Student Activity Heatmap

Purpose: Visualize temporal engagement patterns across the student cohort.
Axes: Days (X) vs. Students (Y), color-coded by action count.
Insight: Identify "cramming" behavior (sudden spikes before deadlines) vs. consistent engagement.

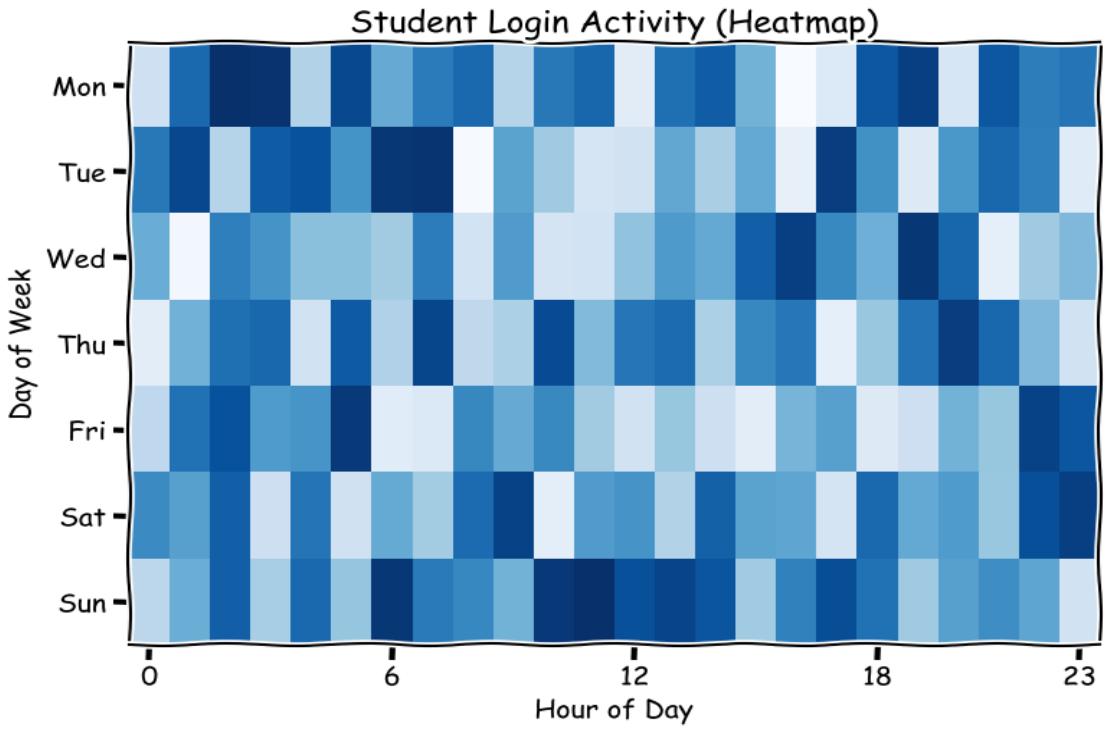

Figure 3.2: Activity Heatmap Mockup (Sketch-style prototype)

3.3.2 Individual Student Risk Timeline

Purpose: Track a single student's risk trajectory across the semester.
Controls: Student selector dropdown.
Insight: Measure the effectiveness of interventions (e.g., risk decrease after a tutor assignment).

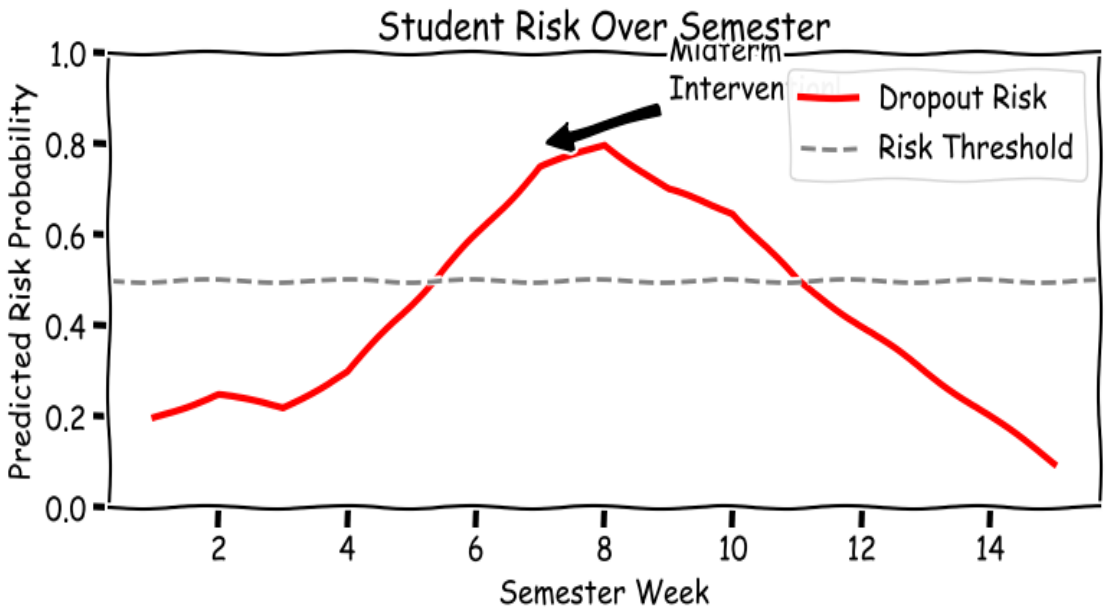

Figure 3.3: Prediction Timeline Mockup - Weekly risk probability evolution

3.3.3 Feature Importance Explorer

Purpose: Understand which behavioral signals drive predictions.
Interactivity: Click a bar to see distribution plots for High vs. Low performers.
Insight: Guide instructional design (e.g., if 'Forum Posts' is highly predictive, emphasize discussions).

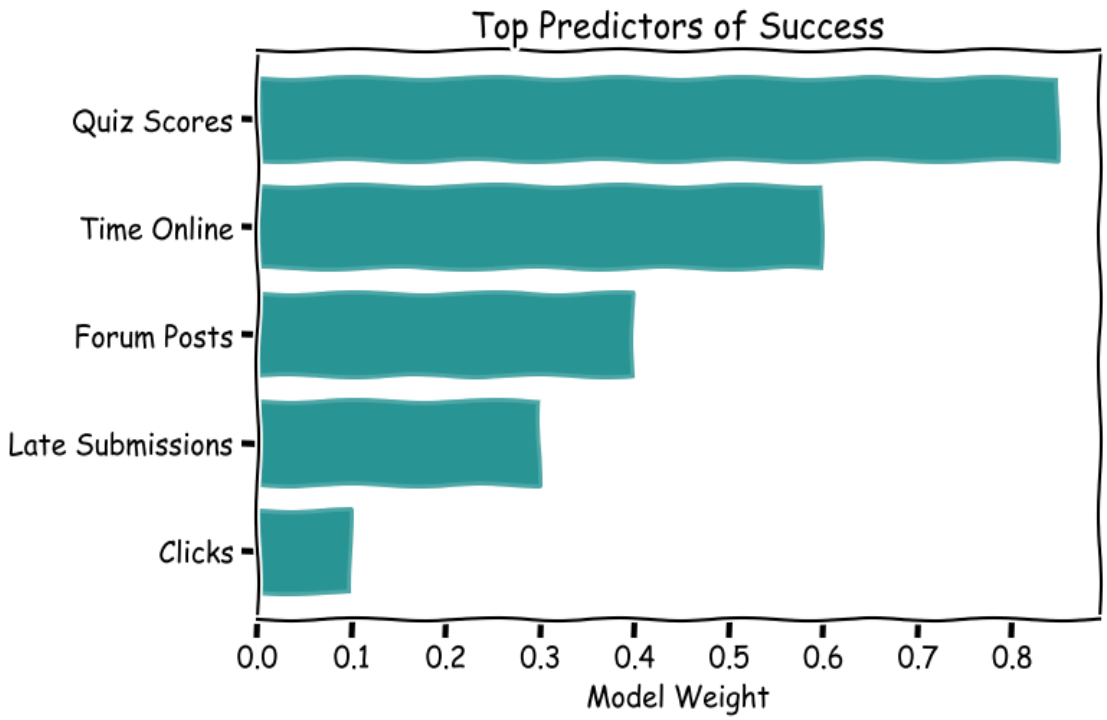

Figure 3.4: Feature Importance Bar Chart Mockup

4. Design Decisions & Rationale

Decision	Rationale	Trade-offs
Sessionization Threshold: 1.5 hours	Educational psychology research indicates typical study blocks range from 60-90 minutes. Gap sig	This may break long, uninterrupted study sessions into multiple sessions. Tradeoff:
Random Forest (vs. Logistic Regression)	Behavioral data exhibits non-linear patterns (e.g., study entropy interacts with total time). RF cap	Less interpretable than LR. RF is a black box. Tradeoff:
Entropy as a Feature	Shannon entropy quantifies the "predictability" of daily activity. Low entropy (e.g., studying	Requires sufficient data points (>7 days) to be stable. Tradeoff:
Weekly Batch (vs. Real-time)	Course dynamics change slowly. Real-time predictions would be noisy and resource-intensive. W	Cannot react to sudden events (e.g., student loses interest).

Table 4.1: Key Design Decisions

5. Algorithm & Methodology

5.1 Feature Engineering Pipeline

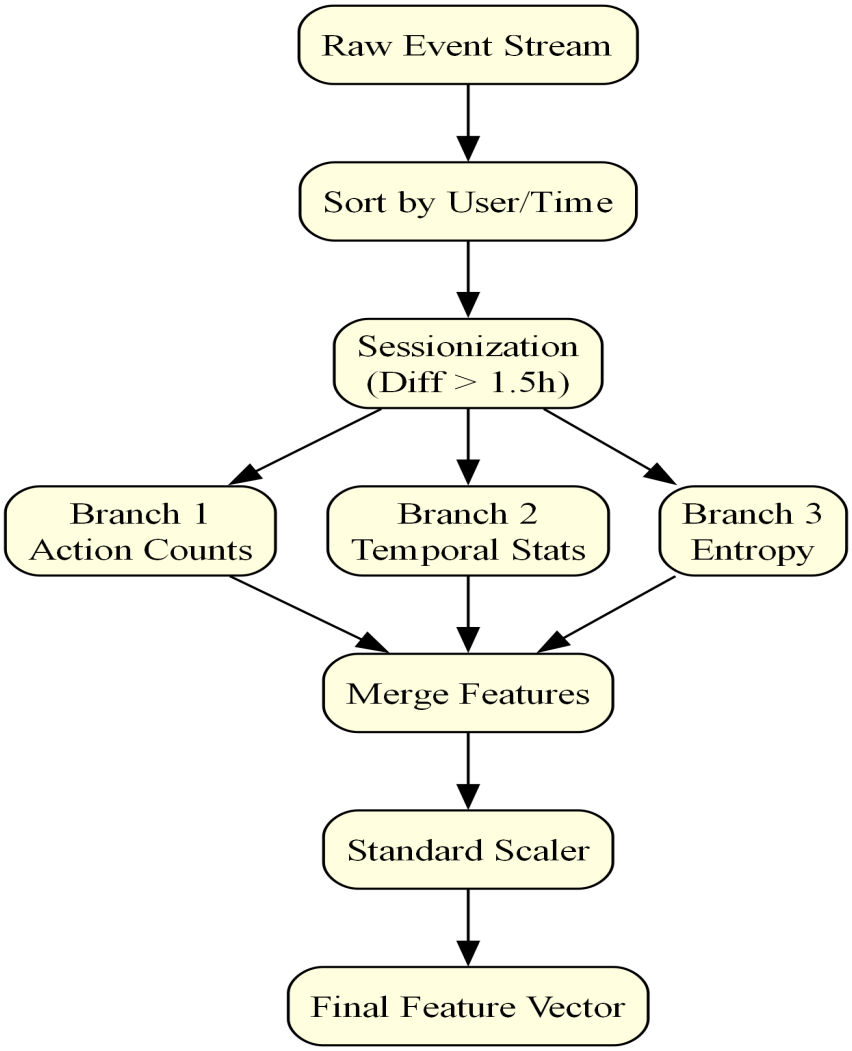

Figure 5.1: Feature Engineering Data Flow

5.2 Feature Categories

The system computes 24 features grouped into four categories:
1. Activity Counts (6 features): Total actions, actions per type (view, post, submit), unique days active.
2. Temporal Statistics (8 features): Median/max session length, avg gap between sessions, weekend activity ratio.
3. Entropy Measures (3 features): Shannon entropy of daily counts, time-of-day distribution entropy, day-of-week entropy.
4. Trend Features (7 features): Week-over-week growth rates, coefficient of variation in daily activity.

5.3 Model Training Protocol

Training Data: Weeks 1-(N-1) for predicting Week N.
Validation: 5-fold stratified cross-validation.
Hyperparameters: n_estimators=200, max_depth=15, min_samples_split=10 (tuned via GridSearchCV).
Class Imbalance: Handled via class_weight='balanced' parameter.
Deployment Threshold: F1-score must exceed 0.65 on validation set before deployment.

5.4 Data Flow Logic

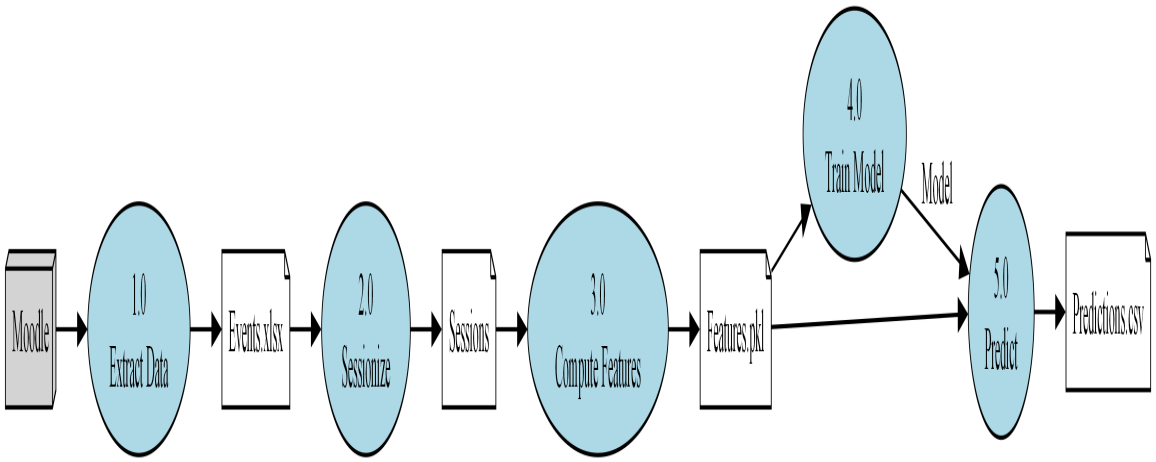

Figure 5.2: Level-1 Data Flow Diagram

6. Security & Data Privacy

The system adheres to FERPA (Family Educational Rights and Privacy Act) regulations and implements defense-in-depth security practices.

- **Pseudonymization:** Student names/emails are replaced with hashed IDs before processing. Mapping tables are stored in an access-controlled database (not in the codebase).
- **Role-Based Access Control (RBAC):** Only course instructors can view predictions for their courses. Admins have read-only access to aggregated metrics.
- **Data Retention:** Raw event logs are purged 90 days post-semester. Aggregated features and predictions are retained for 2 years for longitudinal research (with IRB approval).
- **Encryption:** Data at rest (databases) encrypted via AES-256. Data in transit (API calls) secured via TLS 1.3.

7. Deployment & Operations

The system operates on a **weekly batch schedule** (executed every Sunday at 23:00 UTC). Predictions are generated for the upcoming week and made available to instructors by Monday morning.

7.1 Infrastructure

- **Compute:** AWS EC2 t3.large instance (2 vCPU, 8GB RAM). Scales to c5.xlarge during model retraining.
- **Storage:** Amazon S3 for raw data archives. RDS PostgreSQL for structured predictions/features.
- **Orchestration:** Apache Airflow for pipeline scheduling and monitoring.

7.2 Model Versioning

Models are versioned semantically (e.g., v1.2.3) and stored in an MLflow registry. Each prediction record includes the model_version field to enable A/B testing and rollback capabilities.

8. Testing Strategy

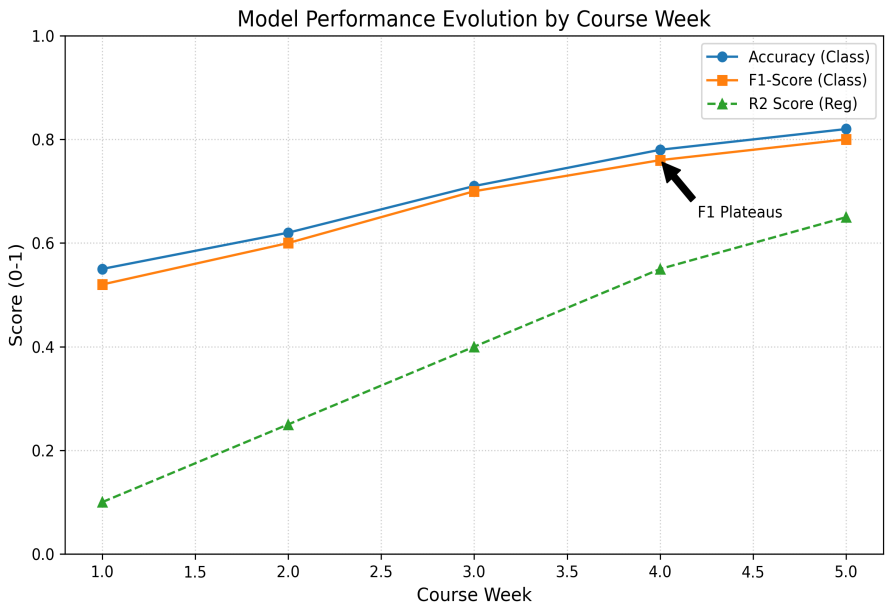

Figure 8.1: Model Performance Evolution (Simulated Data)

8.1 Test Coverage

Level	Scope	Tools
Unit Tests	Individual functions (feature calculators, sessionization)	pytest, coverage.py
Integration Tests	End-to-end pipeline on synthetic data	pytest-integration
Validation Tests	Model performance on held-out semesters	scikit-learn metrics
User Acceptance	Instructor usability testing (dashboard)	Manual + SUS questionnaire

Table 8.1: Testing Pyramid

Deployment Criteria: Validation ensures F1-score > 0.65 and Precision > 0.70 before model deployment. Models failing these thresholds trigger alerts for manual review and potential hyperparameter retuning.

Appendix A: Feature Dictionary

Feature Name	Type	Description	Typical Range
action_cnt	Integer	Total interactions in time window	50-500
session_cnt	Integer	Number of distinct study sessions	5-30
entropy_daily_cnts	Float	Shannon entropy of daily activity distribution	0.5-2.5
median_session_len	Float (min)	Typical duration of a study session	15-90
weekend_ratio	Float	Proportion of activity on Sat/Sun	0.1-0.4
action_growth_rate	Float	Week-over-week % change in activity	-50% to +100%

Appendix B: Configuration Parameters

```
# Core Pipeline Settings SESSION_THRESHOLD = 1.5 # hours MIN_EVENTS_PER_STUDENT = 10
PREDICTION_WEEKS = [2, 3, 4, 5] # Model Hyperparameters RF_N_ESTIMATORS = 200
RF_MAX_DEPTH = 15 RF_MIN_SAMPLES_SPLIT = 10 # Deployment Thresholds MIN_F1_SCORE =
0.65 MIN_PRECISION = 0.70 MIN_TRAINING_SAMPLES = 50
```

Appendix C: Glossary

- **Entropy:** A statistical measure of randomness/unpredictability. In this context, quantifies study schedule consistency.
- **FERPA:** Family Educational Rights and Privacy Act - U.S. law protecting student education records.
- **Pseudonymization:** Replacing identifiable information with artificial identifiers.
- **Random Forest:** An ensemble machine learning algorithm using multiple decision trees.
- **Sessionization:** The process of grouping time-stamped events into meaningful sessions based on temporal gaps.
- **SHAP Values:** SHapley Additive exPlanations - method for explaining individual predictions.